Unit V

Cluster Analysis :Basic Concepts and Methods, Partitioning Methods, Hierarchical Methods : Agglomerative versus.Divisive Hierarchical Clustering, Density-Based Methods: DBSCAN, Grid-Based Methods : STING,Outliers and Outlier Analysis.

INTRODUCTION:

Cluster analysis, also known as clustering, is a method of data mining that groups similar data points together. The goal of cluster analysis is to divide a dataset into groups (or clusters) such that the data points within each group are more similar to each other than to data points in other groups. This process is often used for exploratory data analysis and can help identify patterns or relationships within the data that may not be immediately obvious. There are many different algorithms used for cluster analysis, such as k-means, hierarchical clustering, and density-based clustering. The choice of algorithm will depend on the specific requirements of the analysis and the nature of the data being analyzed.

Cluster Analysis is the process to find similar groups of objects in order to form clusters. It is an unsupervised machine learning-based algorithm that acts on unlabelled data. A group of data points would comprise together to form a cluster in which all the objects would belong to the same group.

The given data is divided into different groups by combining similar objects into a group. This group is nothing but a cluster. A cluster is nothing but a collection of similar data which is grouped together.

For example, consider a dataset of vehicles given in which it contains information about different vehicles like cars, buses, bicycles, etc. As it is unsupervised learning there are no class labels like Cars, Bikes, etc for all the vehicles, all the data is combined and is not in a structured manner.

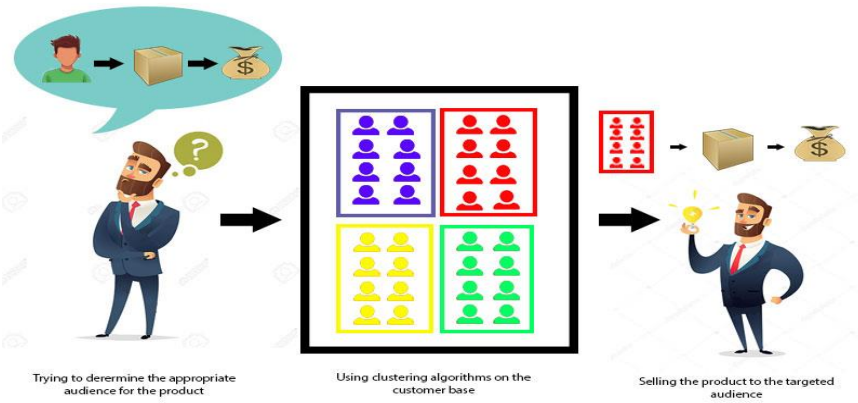
Now our task is to convert the unlabelled data to labelled data and it can be done using clusters.

The main idea of cluster analysis is that it would arrange all the data points by forming clusters like cars cluster which contains all the cars, bikes clusters which contains all the bikes, etc.

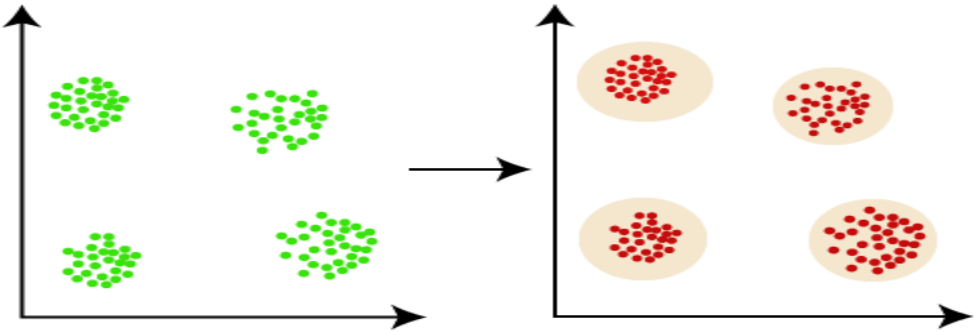
Simply it is the partitioning of similar objects which are applied to unlabelled data.

Clustering is an unsupervised Machine Learning-based Algorithm that comprises a group of data points into clusters so that the objects belong to the same group.

Clustering helps to splits data into several subsets. Each of these subsets contains data similar to each other, and these subsets are called clusters. Now that the data from our customer base is divided into clusters, we can make an informed decision about who we think is best suited for this product.



Let's understand this with an example, suppose we are a market manager, and we have a new tempting product to sell. We are sure that the product would bring enormous profit, as long as it is sold to the right people. So, how can we tell who is best suited for the product from our company's huge customer base?



Showing four clusters formed from the set of unlabeled data

Clustering, falling under the category of **unsupervised machine learning**, is one of the problems that machine learning algorithms solve.

Clustering only utilizes input data, to determine patterns, anomalies, or similarities in its input data.

A good clustering algorithm aims to obtain clusters whose:

- The intra-cluster similarities are high, It implies that the data present inside the cluster is similar to one another.
- The inter-cluster similarity is low, and it means each cluster holds data that is not similar to other data.

What is a Cluster?

- A cluster is a subset of similar objects
- A subset of objects such that the distance between any of the two objects in the cluster is less than the distance between any object in the cluster and any object that is not located inside it.
- A connected region of a multidimensional space with a comparatively high density of objects.

What is clustering in Data Mining?

- Clustering is the method of converting a group of abstract objects into classes of similar objects.
- Clustering is a method of partitioning a set of data or objects into a set of significant subclasses called clusters.
- It helps users to understand the structure or natural grouping in a data set and used either as a stand-alone instrument to get a better insight into data distribution or as a pre-processing step for other algorithms

Types Of Data Used In Cluster Analysis Are:

- Interval-Scaled variables
- Binary variables
- Nominal, Ordinal, and Ratio variables
- Variables of mixed types

Types Of Data Structures

First of all, let us know what types of data structures are widely used in cluster analysis.

We shall know the types of data that often occur in cluster analysis and how to preprocess them for such analysis.
Suppose that a data set to be clustered contains n objects, which may represent persons, houses, documents, countries, and so on.
Main memory-based clustering algorithms typically operate on either of the following two data structures.

Types of data structures in cluster analysis are

- Data Matrix (or object by variable structure)
- Dissimilarity Matrix (or object by object structure)

Data Matrix

This represents n objects, such as persons, with p variables (also called measurements or attributes), such as age, height, weight, gender, race and so on. The structure is in the form of a relational table, or n-by-p matrix (n objects x p variables)

The Data Matrix is often called a two-mode matrix since the rows and columns of this represent the different entities.

Dissimilarity Matrix

This stores a collection of proximities that are available for all pairs of n objects. It is often represented by a n – by – n table, where d(i,j) is the measured difference or dissimilarity between objects i and j. In general, d(i,j) is a non-negative number that is close to 0 when objects i and j are higher similar or "near" each other and becomes larger the more they differ. Since d(i,j) = d(j,i) and d(i,i) =0, we have the matrix in figure.

This is also called as one mode matrix since the rows and columns of this represent the same entity.

Types Of Data In Cluster Analysis Are:

Interval-Scaled Variables

Interval-scaled variables are continuous measurements of a roughly linear scale.
Typical examples include weight and height, latitude and longitude coordinates (e.g., when clustering houses), and weather temperature.
The measurement unit used can affect the clustering analysis. For example, changing measurement units from meters to inches for height, or from kilograms to pounds for weight, may lead to a very different clustering structure.

In general, expressing a variable in smaller units will lead to a larger range for that variable, and thus a larger effect on the resulting clustering structure.

To help avoid dependence on the choice of measurement units, the data should be standardized. Standardizing measurements attempts to give all variables an equal weight.

This is especially useful when given no prior knowledge of the data. However, in some applications, users may intentionally want to give more weight to a certain set of variables than to others.

For example, when clustering basketball player candidates, we may prefer to give more weight to the variable height.

Binary Variables

A binary variable is a variable that can take only 2 values.

For example, generally, gender variables can take 2 variables male and female.

Contingency Table For Binary Data
Let us consider binary values 0 and 1

Let p=a+b+c+d

Simple matching coefficient (invariant, if the binary variable is symmetric):

Jaccard coefficient (noninvariant if the binary variable is asymmetric):
Nominal or Categorical Variables

A generalization of the binary variable in that it can take more than 2 states, e.g., red, yellow, blue, green.

Method 1: Simple matching

The dissimilarity between two objects i and j can be computed based on the simple matching.

m: Let m be no of matches (i.e., the number of variables for which i and j are in the same state).

p: Let p be total no of variables.

Method 2: use a large number of binary variables
Creating a new binary variable for each of the M nominal states.
Ordinal Variables

An ordinal variable can be discrete or continuous.

In this order is important, e.g., rank.

It can be treated like interval-scaled

By replacing xif by their rank,

By mapping the range of each variable onto [0, 1] by replacing the i-th object in the f-th variable by,

Then compute the dissimilarity using methods for interval-scaled variables.

Ratio-Scaled Intervals

Ratio-scaled variable: It is a positive measurement on a nonlinear scale, approximately at an exponential scale, such as Ae^Bt or A^e-Bt.

Methods:

- First, treat them like interval-scaled variables — not a good choice! (why?)
- Then apply logarithmic transformation i.e.y = log(x)
- Finally, treat them as continuous ordinal data treat their rank as interval-scaled.

Variables Of Mixed Type

A database may contain all the six types of variables
symmetric binary, asymmetric binary, nominal, ordinal, interval, and ratio.

And those combinedly called as mixed-type variables.

Partitioning Method: This clustering method classifies the information into multiple groups based on the characteristics and similarity of the data. Its the data analysts to specify the number of clusters that has to be generated for the clustering methods. In the partitioning method when database(D) that contains multiple(N) objects then the partitioning method constructs user-specified(K) partitions of the data in which each partition represents a cluster and a particular region. There are many algorithms that come under partitioning method some of the popular ones are K-Mean, PAM(K-Medoids), CLARA algorithm (Clustering Large Applications) etc. In this article, we will be seeing the working of K Mean algorithm in detail.

K-Mean (A centroid based Technique): The K means algorithm takes the input parameter K from the user and partitions the dataset containing N objects into K clusters so that resulting similarity among the data objects inside the group (intracluster) is high but the similarity of data objects with the data objects from outside the cluster is low (intercluster). The similarity of the cluster is determined with respect to the mean value of the cluster. It is a type of square error algorithm. At the start randomly k objects from the dataset are chosen in which each of the objects represents a cluster mean(centre). For the rest of the data objects, they are assigned to the nearest cluster based on their distance from the cluster mean. The new mean of each of the cluster is then calculated with the added data objects.

Algorithm:

K mean:

```
Input:
K: The number of clusters in which the dataset has to be divided
D: A dataset containing N number of objects

Output:
A dataset of K clusters
```

Method:

1. Randomly assign K objects from the dataset(D) as cluster centres(C)

2. (Re) Assign each object to which object is most similar based upon mean values.

3. Update Cluster means, i.e., Recalculate the mean of each cluster with the updated values.

4. Repeat Step 2 until no change occurs.

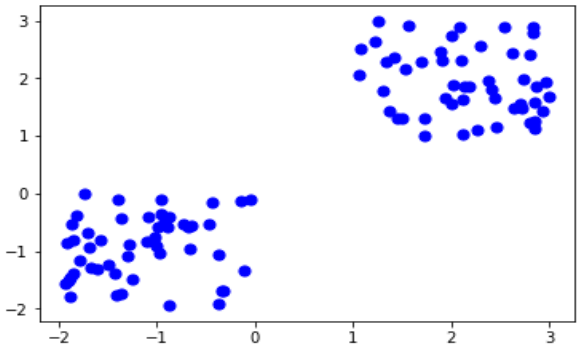


Figure – K-mean Clustering

Flowchart:

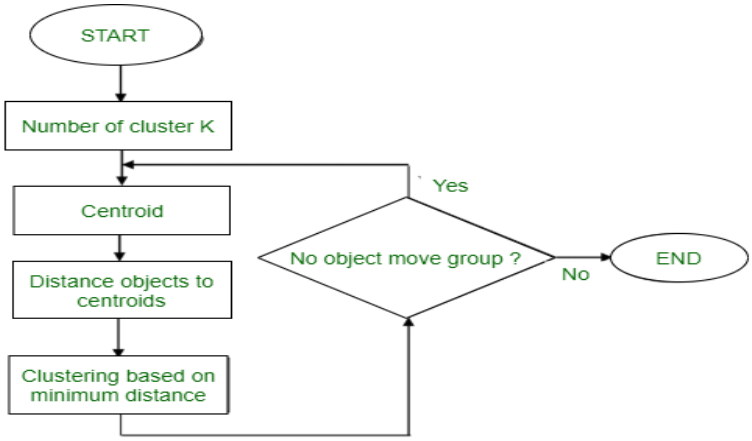


Figure – K-mean Clustering

Example: Suppose we want to group the visitors to a website using just their age as follows:

```
16, 16, 17, 20, 20, 21, 21, 22, 23, 29, 36, 41, 42, 43, 44, 45, 61, 62, 66
```

Initial Cluster:

```
K=2
Centroid(C1) = 16 [16]
Centroid(C2) = 22 [22]
```

Note: These two points are chosen randomly from the dataset.

Iteration-1:

```
C1 = 16.33 [16, 16, 17]
C2 = 37.25 [20, 20, 21, 21, 22, 23, 29, 36, 41, 42, 43, 44, 45, 61, 62, 66]
```

Iteration-2:

```
C1 = 19.55 [16, 16, 17, 20, 20, 21, 21, 22, 23]
C2 = 46.90 [29, 36, 41, 42, 43, 44, 45, 61, 62, 66]
```

Iteration-3:

```
C1 = 20.50 [16, 16, 17, 20, 20, 21, 21, 22, 23, 29]
C2 = 48.89 [36, 41, 42, 43, 44, 45, 61, 62, 66]
```

Iteration-4:

```
C1 = 20.50 [16, 16, 17, 20, 20, 21, 21, 22, 23, 29]
C2 = 48.89 [36, 41, 42, 43, 44, 45, 61, 62, 66]
```

No change Between Iteration 3 and 4, so we stop. Therefore we get the clusters **(16-29)** and **(36-66)** as 2 clusters we get using K Mean Algorithm.

K-Medoids clustering-Theoretical Explanation

K-Medoids and K-Means are two types of clustering mechanisms in Partition Clustering. First, Clustering is the process of breaking down an abstract group of data points/ objects into classes of similar objects such that all the objects in one cluster have similar traits. , a group of n objects is broken down into k number of clusters based on their similarities.

Two statisticians, Leonard Kaufman, and Peter J. Rousseeuw came up with this method. This tutorial explains what K-Medoids do, their applications, and the difference between K-Means and K-Medoids.

K-medoids is an unsupervised method with unlabelled data to be clustered. It is an improvised version of the K-Means algorithm mainly designed to deal with outlier data sensitivity. Compared to other partitioning algorithms, the algorithm is simple, fast, and easy to implement.

The partitioning will be carried on such that:

1. Each cluster must have at least one object
2. An object must belong to only one cluster

K-Medoids:

Medoid: A Medoid is a point in the cluster from which the sum of distances to other data points is minimal.

(or)

A Medoid is a point in the cluster from which dissimilarities with all the other points in the clusters are minimal.

Instead of centroids as reference points in K-Means algorithms, the K-Medoids algorithm takes a Medoid as a reference point.

There are three types of algorithms for K-Medoids Clustering:

1. PAM (Partitioning Around Clustering)
2. CLARA (Clustering Large Applications)
3. CLARANS (Randomized Clustering Large Applications)

PAM is the most powerful algorithm of the three algorithms but has the disadvantage of time complexity. The following K-Medoids are performed using PAM. In the further parts, we'll see what CLARA and CLARANS are.

Algorithm:

Given the value of k and unlabelled data:

1. Choose k number of random points from the data and assign these k points to k number of clusters. These are the initial medoids.
2. For all the remaining data points, calculate the distance from each medoid and assign it to the cluster with the nearest medoid.
3. Calculate the total cost (Sum of all the distances from all the data points to the medoids)

4. Select a random point as the new medoid and swap it with the previous medoid. Repeat 2 and 3 steps.
5. If the total cost of the new medoid is less than that of the previous medoid, make the new medoid permanent and repeat step 4.
6. If the total cost of the new medoid is greater than the cost of the previous medoid, undo the swap and repeat step 4.
7. The Repetitions have to continue until no change is encountered with new medoids to classify data points.

CLARA:

It is an extension to PAM to support Medoid clustering for large data sets. This algorithm selects **data samples from the data set, applies Pam on each sample, and outputs the best Clustering out of these samples**. This is more effective than PAM. We should ensure that the selected samples aren't biased as they affect the Clustering of the whole data.

CLARANS:

This algorithm selects a sample of neighbors to examine instead of selecting samples from the data set. In every step, it examines the neighbors of every node. The time complexity of this algorithm is $O(n^2)$, and this is the best and most efficient Medoids algorithm of all.

A **Hierarchical clustering** method works via grouping data into a tree of clusters. Hierarchical clustering begins by treating every data point as a separate cluster. Then, it repeatedly executes the subsequent steps:

1. Identify the 2 clusters which can be closest together, and

2. Merge the 2 maximum comparable clusters. We need to continue these steps until all the clusters are merged together.

In Hierarchical Clustering, the aim is to produce a hierarchical series of nested clusters. A diagram called **Dendrogram** (A Dendrogram is a tree-like diagram that statistics the sequences of merges or splits) graphically represents this hierarchy and is an inverted tree that describes the order in which factors are merged (bottom-up view) or clusters are broken up (top-down view).

What is Hierarchical Clustering?

Hierarchical clustering is a method of cluster analysis in data mining that creates a hierarchical representation of the clusters in a dataset. The method starts by treating each data point as a separate cluster and then iteratively combines the closest clusters until a stopping criterion is reached. The result of hierarchical clustering is a tree-like structure, called a dendrogram, which illustrates the hierarchical relationships among the clusters.

Hierarchical clustering has several advantages over other clustering methods

- The ability to handle non-convex clusters and clusters of different sizes and densities.

- The ability to handle missing data and noisy data.

- The ability to reveal the hierarchical structure of the data, which can be useful for understanding the relationships among the clusters.

Drawbacks of Hierarchical Clustering

- The need for a criterion to stop the clustering process and determine the final number of clusters.

- The computational cost and memory requirements of the method can be high, especially for large datasets.

- The results can be sensitive to the initial conditions, linkage criterion, and distance metric used. In summary, Hierarchical clustering is a method of data mining that groups similar data points into clusters by creating a hierarchical structure of the clusters.

- This method can handle different types of data and reveal the relationships among the clusters. However, it can have high computational cost and results can be sensitive to some conditions.

Types of Hierarchical Clustering

Basically, there are two types of hierarchical Clustering:

1. Agglomerative Clustering

2. Divisive clustering

1. Agglomerative Clustering

Initially consider every data point as an **individual** Cluster and at every step, merge the nearest pairs of the cluster. (It is a bottom-up method). At first, every dataset is considered an individual entity or cluster. At every iteration, the clusters merge with different clusters until one cluster is formed.

The algorithm for Agglomerative Hierarchical Clustering is:

- Calculate the similarity of one cluster with all the other clusters (calculate proximity matrix)

- Consider every data point as an individual cluster

- Merge the clusters which are highly similar or close to each other.

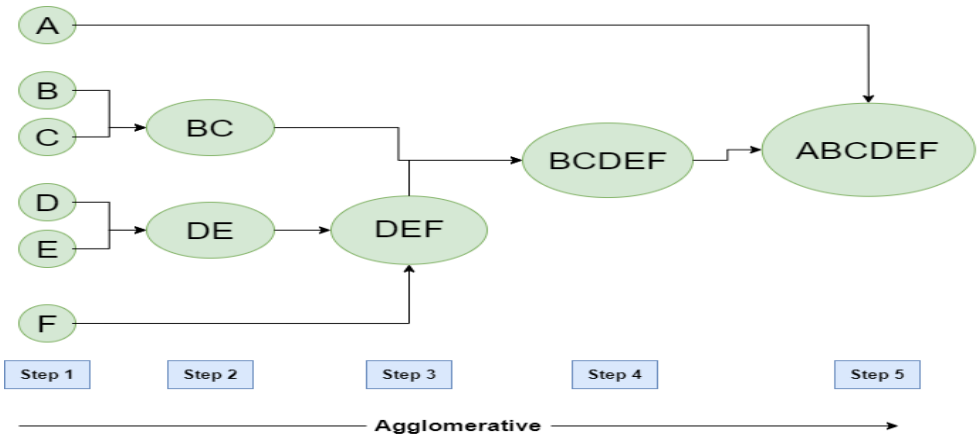
- Recalculate the proximity matrix for each cluster

- Repeat Steps 3 and 4 until only a single cluster remains.

Let's see the graphical representation of this algorithm using a dendrogram.

Note: This is just a demonstration of how the actual algorithm works no calculation has been performed below all the proximity among the clusters is assumed.

Let's say we have six data points **A, B, C, D, E, and F**.



- **Step-1:** Consider each alphabet as a single cluster and calculate the distance of one cluster from all the other clusters.

- **Step-2:** In the second step comparable clusters are merged together to form a single cluster. Let's say cluster (B) and cluster (C) are very similar to each other therefore we merge them in the second step similarly to cluster (D) and (E) and at last, we get the clusters [(A), (BC), (DE), (F)]

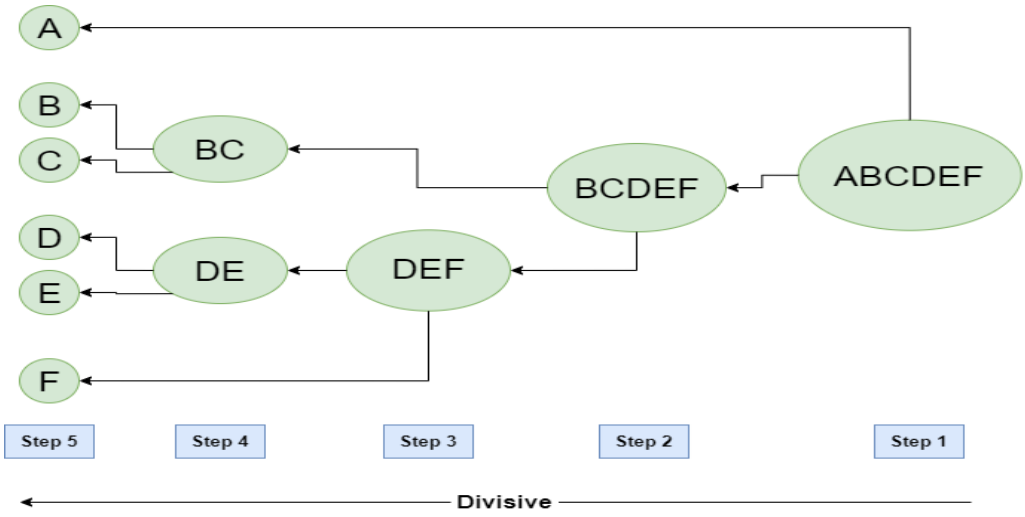
- **Step-3:** We recalculate the proximity according to the algorithm and merge the two nearest clusters([(DE), (F)]) together to form new clusters as [(A), (BC), (DEF)]

- **Step-4:** Repeating the same process; The clusters DEF and BC are comparable and merged together to form a new cluster. We're now left with clusters [(A), (BCDEF)].

- **Step-5:** At last, the two remaining clusters are merged together to form a single cluster [(ABCDEF)].

2. Divisive Hierarchical clustering

We can say that Divisive Hierarchical clustering is precisely the **opposite** of Agglomerative Hierarchical clustering. In Divisive Hierarchical clustering, we take into account all of the data points as a single cluster and in every iteration, we separate the data points from the clusters which aren't comparable. In the end, we are left with N clusters.



Divisive Hierarchical clustering

BIRCH in Data Mining

BIRCH (balanced iterative reducing and clustering using hierarchies) is an unsupervised data mining algorithm that performs hierarchical clustering over large data sets. With modifications, it can also be used to accelerate k-means clustering and Gaussian mixture modeling with the expectation-maximization algorithm. An advantage of BIRCH is its ability to incrementally and dynamically cluster incoming, multi-dimensional metric data points to produce the best quality clustering for a given set of resources (memory and time constraints). In most cases, BIRCH only requires a single scan of the database.

Its inventors claim BIRCH to be the "first clustering algorithm proposed in the database area to handle 'noise' (data points that are not part of the underlying pattern) effectively", beating DBSCAN by two months. The BIRCH algorithm received the SIGMOD 10 year test of time award in 2006.

Basic clustering algorithms like K means and agglomerative clustering are the most commonly used clustering algorithms. But when performing clustering on very large datasets, BIRCH and DBSCAN are the advanced clustering algorithms useful for performing precise clustering on large datasets. Moreover, BIRCH is very useful because of its easy implementation. BIRCH is a clustering algorithm that clusters the dataset first in small summaries, then after small summaries get clustered. It does not directly cluster the dataset. That is why BIRCH is often used with other clustering algorithms; after making the summary, the summary can also be clustered by other clustering algorithms.

It is provided as an alternative to MinibatchKMeans. It converts data to a tree data structure with the centroids being read off the leaf. And these centroids can be the final cluster centroid or the input for other cluster algorithms like Agglomerative Clustering.

Problem with Previous Clustering Algorithm

Previous clustering algorithms performed less effectively over very large databases and did not adequately consider the case wherein a dataset was too large to fit in main memory. Furthermore, most of BIRCH's predecessors inspect all data points (or all currently existing clusters) equally for each clustering decision. They do not perform heuristic weighting based on the distance between these data points. As a result, there was a lot of overhead maintaining high clustering quality while minimizing the cost of additional IO (input/output) operations.

Stages of BIRCH

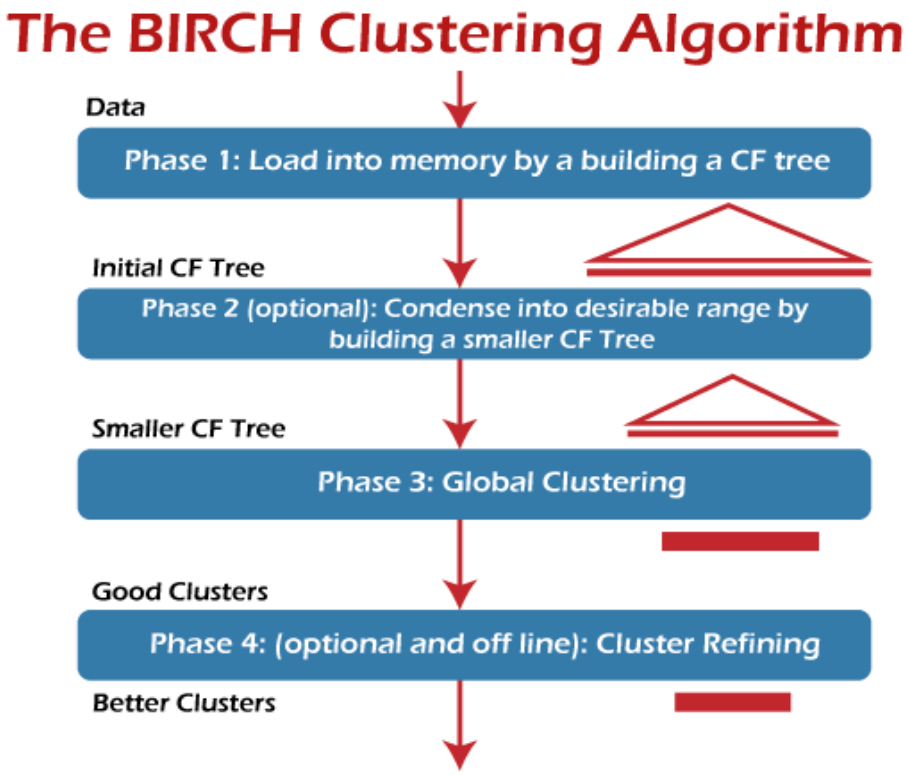
BIRCH is often used to complement other clustering algorithms by creating a summary of the dataset that the other clustering algorithm can now use. However, BIRCH has one major drawback it can only process metric attributes. A metric attribute is an attribute whose values can be represented in Euclidean space, i.e., no categorical attributes should be present. The BIRCH clustering algorithm consists of two stages:

1. **Building the CF Tree:** BIRCH summarizes large datasets into smaller, dense regions called Clustering Feature (CF) entries. Formally, a Clustering Feature entry is defined as an ordered triple (N, LS, SS) where 'N' is the number of data points in the cluster, 'LS' is the linear sum of the data points, and 'SS' is the squared sum of the data points in the cluster. A CF entry can be composed of other CF entries. Optionally, we can condense this initial CF tree into a smaller CF.
2. **Global Clustering:** Applies an existing clustering algorithm on the leaves of the CF tree. A CF tree is a tree where each leaf node contains a sub-cluster. Every entry in a CF tree contains a pointer to a child node, and a CF entry made up of the sum of CF entries in the child nodes. Optionally, we can refine these clusters.

Due to this two-step process, BIRCH is also called ***Two-Step Clustering.***

Algorithm

The tree structure of the given data is built by the BIRCH algorithm called the Clustering feature tree (CF tree). This algorithm is based on the CF (clustering features) tree. In addition, this algorithm uses a tree-structured summary to create clusters.



In context to the CF tree, the algorithm compresses the data into the sets of CF nodes. Those nodes that have several sub-clusters can be called CF subclusters. These CF subclusters are situated in no-terminal CF nodes.

The CF tree is a height-balanced tree that gathers and manages clustering features and holds necessary information of given data for further hierarchical clustering. This prevents the need to work with whole data given as input. The tree cluster of data points as CF is represented by three numbers (N, LS, SS).

- N = number of items in subclusters
- LS = vector sum of the data points
- SS = sum of the squared data points

There are mainly four phases which are followed by the algorithm of BIRCH.

- Scanning data into memory.
- Condense data (resize data).
- Global clustering.
- Refining clusters.

Two of them (resize data and refining clusters) are optional in these four phases. They come in the process when more clarity is required. But scanning data is just like loading data into a model. After loading the data, the algorithm scans the whole data and fits them into the CF trees.

In condensing, it resets and resizes the data for better fitting into the CF tree. In global clustering, it sends CF trees for clustering using existing clustering algorithms. Finally, refining fixes the problem of CF trees where the same valued points are assigned to different leaf nodes.

Cluster Features

BIRCH clustering achieves its high efficiency by clever use of a small set of summary statistics to represent a larger set of data points. These summary statistics constitute a CF and represent a sufficient substitute for the actual data for clustering purposes.

A CF is a set of three summary statistics representing a set of data points in a single cluster. These statistics are as follows:

- Count [The number of data values in the cluster]
- Linear Sum [The sum of the individual coordinates. This is a measure of the location of the cluster]
- Squared Sum [The sum of the squared coordinates. This is a measure of the spread of the cluster]

NOTE: The linear sum and the squared sum are equivalent to the mean and variance of the data point.

CF Tree

The building process of the CF Tree can be summarized in the following steps, such as:

Step 1: For each given record, BIRCH compares the location of that record with the location of each CF in the root node, using either the linear sum or the mean of the CF. BIRCH passes the incoming record to the root node CF closest to the incoming record.

Step 2: The record then descends down to the non-leaf child nodes of the root node CF selected in step 1. BIRCH compares the location of the record with the location of each non-leaf CF. BIRCH passes the incoming record to the non-leaf node CF closest to the incoming record.

Step 3: The record then descends down to the leaf child nodes of the non-leaf node CF selected in step 2. BIRCH compares the location of the record with the location of each leaf. BIRCH tentatively passes the incoming record to the leaf closest to the incoming record.

Step 4: Perform one of the below points (i) or (ii):

- If the radius of the chosen leaf, including the new record, does not exceed the threshold T, then the incoming record is assigned to that leaf. The leaf and its parent CF's are updated to account for the new data point.

- If the radius of the chosen leaf, including the new record, exceeds the Threshold T, then a new leaf is formed, consisting of the incoming record only. The parent CFs is updated to account for the new data point.

If step 4(ii) is executed, and the maximum L leaves are already in the leaf node, the leaf node is split into two leaf nodes. If the parent node is full, split the parent node, and so on. The most distant leaf node CFs are used as leaf node seeds, with the remaining CFs being assigned to whichever leaf node is closer. Note that the radius of a cluster may be calculated even without knowing the data points, as long as we have the count n, the linear sum LS, and the squared sum SS. This allows BIRCH to evaluate whether a given data point belongs to a particular sub-cluster without scanning the original data set.

Clustering the Sub-Clusters

Once the CF tree is built, any existing clustering algorithm may be applied to the sub-clusters (the CF leaf nodes) to combine these sub-clusters into clusters. The task of clustering becomes much easier as the number of sub-clusters is much less than the number of data points. When a new data value is added, these statistics may be easily updated, thus making the computation more efficient.

Parameters of BIRCH

There are three parameters in this algorithm that needs to be tuned. Unlike K-means, the optimal number of clusters (k) need not be input by the user as the algorithm determines them.

- **Threshold:** Threshold is the maximum number of data points a sub-cluster in the leaf node of the CF tree can hold.
- **branching_factor:** This parameter specifies the maximum number of CF sub-clusters in each node (internal node).
- **n_clusters:** The number of clusters to be returned after the entire BIRCH algorithm is complete, i.e., the number of clusters after the final clustering step. The final clustering step is not performed if set to none, and intermediate clusters are returned.

ROCK stands for Robust Clustering using links. It is a hierarchical clustering algorithm that analyze the concept of links (the number of common neighbours among two objects) for data with categorical attributes. It display that such distance data cannot lead to high-quality clusters when clustering categorical information.

Moreover, most clustering algorithms create only the similarity among points when clustering i.e., at each step, points that are combined into a single cluster. This "localized" method is prone to bugs. For instance, two distinct clusters can have a few points or outliers that are near; thus, relying on the similarity among points to create clustering decisions can generate the two clusters to be combined.

ROCK takes a more global method to clustering by treating the neighborhoods of single pairs of points. If two similar points also have same neighborhoods, thus the two points likely belong to the similar cluster and so can be combined.

There are two points, p_i and p_j, are neighbors if $sim(p_i, p_j) \geq \theta$, where sim is a similarity function and θ is a user-specified threshold. It can select sim to be a distance metric or even a nonmetric that is normalized so that its values fall among 0 and 1, with higher values denoting that the points are more same.

The number of connection between p_i and p_j is represented as the number of common neighbors between p_i and p_j. If the number of links between two points is high, then it is more likely that they belong to the similar cluster. By treating neighboring data points in the relationship among individual group of points, ROCK is powerful than standard clustering methods that target only on point similarity.

An instance of data including categorical attributes is market basket information. Such data includes a database of transactions, where each transaction is a group of items. Transactions are treated data with Boolean attributes, each corresponding to a single item, including bread or cheese.

- In the data for a transaction, the attribute corresponding to an item is correct if the transaction include the item; otherwise, it is false. There are several data sets with categorical attributes can be managed in a same manner. ROCK's terms of neighbors and links are the same between two "points" or transactions, T_i and T_j, is represented with the Jaccard coefficient as
  - $sim(T_i,T_j)=|T_i \cap T_j||T_i \cup T_j|$
- ROCK first produce a sparse graph from a given data similarity matrix utilizing a similarity threshold and the approach of shared neighbors. It can implements agglomerative hierarchical clustering on the sparse graph. A goodness measure can compute the clustering. Random sampling can be used for scaling up to high data sets.
- The worst-case time complexity of ROCK is $O(n^2 + nm_m m_a + n^2 \log_n)$ where m_m and m_a the maximum and average number of neighbors, accordingly, are and n is the number of objects.

What is a Chameleon?

Chameleon is a hierarchical clustering algorithm that uses dynamic modeling to decide the similarity among pairs of clusters. It was changed based on the observed weaknesses of two hierarchical clustering algorithms such as ROCK and CURE.

ROCK and related designs emphasize cluster interconnectivity while neglecting data regarding cluster proximity. CURE and related design consider cluster proximity yet neglect cluster interconnectivity. In Chameleon, cluster similarity is assessed depending on how well-connected objects are inside a cluster and on the proximity of clusters. Especially, two clusters are combined if their interconnectivity is high and they are close together.

It does not base on a static, user-supplied model and can automatically adapt to the internal features of the clusters being combined. The merge process supports the discovery of natural and homogeneous clusters and is used for all types of data considering a similarity function can be defined.

Chameleon needs the k-nearest-neighbor graph technique to make a sparse graph, where each vertex of the graph defines a data object, and there exists an edge among two vertices (objects) if one object is between the k-most-similar objects of the other. The edges are weighted to reflect the similarity among objects.

Chameleon uses a graph partitioning algorithm to partition the k-nearest-neighbor graph into a large number of relatively small subclusters. It can use an agglomerative hierarchical clustering algorithm that repeatedly merges subclusters based on their similarity. It can determine the pairs of most similar subclusters, it takes into account both the interconnectivity as well as the closeness of the clusters.

The k-nearest-neighbor graph captures the approach of neighborhood dynamically: the neighborhood radius of an object is decided by the density of the region in which the object resides. In a dense area, the neighborhood is represented narrowly. In a sparse region, it is represented more widely.

This influence results in more natural clusters, in comparison with density-based methods like DBSCAN that instead use a worldwide neighborhood. Furthermore, the density of the region is recorded as the weight of the edges. Especially, the edges of a dense region tend to weigh more than that of a sparse region.

The graph-partitioning algorithm partitions the k-nearest-neighbor graph such that it makes smaller the edge cut. That is, cluster C is subdivided into sub-clusters $C_i$and C_j to minimize the weight of the edges that can be cut should C be bisected into C_i and C_j . Edge cut is indicated EC (C_i, C_j)and determines the absolute interconnectivity between cluster C_i and C_j .

Density-based clustering in data mining

Density-based clustering refers to a method that is based on local cluster criterion, such as density connected points. In this tutorial, we will discuss density-based clustering with examples.

What is Density-based clustering?

Density-Based Clustering refers to one of the most popular unsupervised learning methodologies used in model building and machine learning algorithms. The data points in the region separated by two clusters of low point density are considered as noise. The surroundings with a radius ε of a given object are known as the ε neighborhood of the object. If the ε neighborhood of the object comprises at least a minimum number, MinPts of objects, then it is called a core object.

Density-Based Clustering - Background

There are two different parameters to calculate the density-based clustering

E_{PS}: It is considered as the maximum radius of the neighborhood.

MinPts: MinPts refers to the minimum number of points in an Eps neighborhood of that point.

NEps (i) : { k belongs to D and dist (i,k) < = Eps}

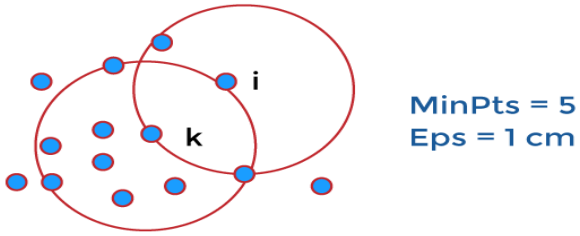
Directly density reachable:

A point i is considered as the directly density reachable from a point k with respect to Eps, MinPts if

i belongs to NEps(k)

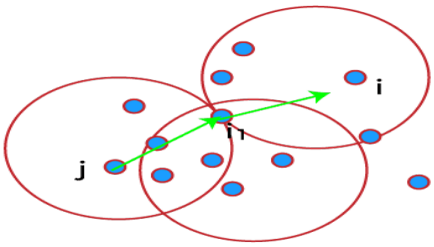
Core point condition:

NEps (k) >= MinPts



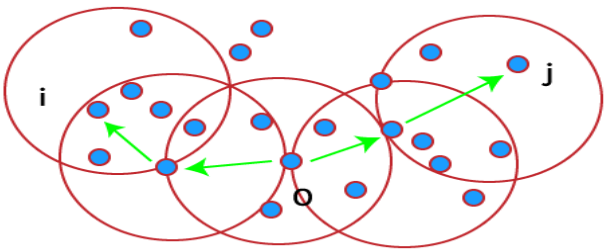
Density reachable:

A point denoted by i is a density reachable from a point j with respect to Eps, MinPts if there is a sequence chain of a point i1,…., in, i1 = j, pn = i such that i_i + 1 is directly density reachable from i_i.



Density connected:

A point i refers to density connected to a point j with respect to Eps, MinPts if there is a point o such that both i and j are considered as density reachable from o with respect to Eps and MinPts.



Working of Density-Based Clustering

Suppose a set of objects is denoted by D', we can say that an object I is directly density reachable form the object j only if it is located within the ε neighborhood of j, and j is a core object.

An object i is density reachable form the object j with respect to ε and MinPts in a given set of objects, D' only if there is a sequence of object chains point i1,…., in, i1 = j, pn = i such that $i_i + 1$ is directly density reachable from i_i with respect to ε and MinPts.

An object i is density connected object j with respect to ε and MinPts in a given set of objects, D' only if there is an object o belongs to D such that both point i and j are density reachable from o with respect to ε and MinPts.

Major Features of Density-Based Clustering

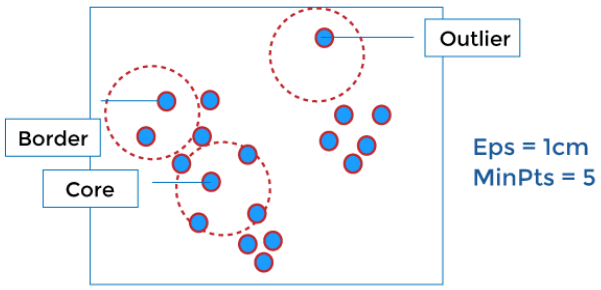
The primary features of Density-based clustering are given below.

- It is a scan method.
- It requires density parameters as a termination condition.
- It is used to manage noise in data clusters.
- Density-based clustering is used to identify clusters of arbitrary size.

Density-Based Clustering Methods

DBSCAN stands for Density-Based Spatial Clustering of Applications with Noise. It depends on a density-based notion of cluster. It also identifies clusters of arbitrary size in the spatial database with outliers.

DBSCAN stands for Density-Based Spatial Clustering of Applications with Noise. It depends on a density-based notion of cluster. It also identifies clusters of arbitrary size in the spatial database with outliers.



OPTICS

OPTICS stands for Ordering Points To Identify the Clustering Structure. It gives a significant order of database with respect to its density-based clustering structure. The order of the cluster comprises information equivalent to the density-based clustering related to a long range of parameter settings. OPTICS methods are beneficial for both automatic and interactive cluster analysis, including determining an intrinsic clustering structure.

DENCLUE

Density-based clustering by Hinnebirg and Kiem. It enables a compact mathematical description of arbitrarily shaped clusters in high dimension state of data, and it is good for data sets with a huge amount of noise.

Grid-Based Method in Data Mining:

In Grid-Based Methods, the space of instance is divided into a grid structure. Clustering techniques are then applied using the Cells of the grid, instead of individual data points, as the base units. The biggest advantage of this method is to improve the processing time.

STING – Statistical Information Grid in Data Mining

STING is a Grid-Based Clustering Technique. In STING, the dataset is recursively divided in a hierarchical manner. After the dataset, each cell is divided into a different number of cells. And after the cell, the statistical measures of the cell are collected, which helps answer the query as quickly as possible.

Statistical Information Grid(STING):

A STING is a grid-based clustering technique. It uses a multidimensional grid data structure that quantifies space into a finite number of cells. Instead of focusing on data points, it focuses on the value space surrounding the data points.

In STING, the spatial area is divided into rectangular cells and several levels of cells at different resolution levels. High-level cells are divided into several low-level cells.

In STING Statistical Information about attributes in each cell, such as mean, maximum, and minimum values, are precomputed and stored as statistical parameters. These statistical parameters are useful for query processing and other data analysis tasks.

The statistical parameter of higher-level cells can easily be computed from the parameters of the lower-level cells.

How STING Work:

Step 1: Determine a layer, to begin with.

Step 2: For each cell of this layer, it calculates the confidence interval or estimated range of probability that this is cell is relevant to the query.

Step 3: From the interval calculate above, it labels the cell as relevant or not relevant.

Step 4: If this layer is the bottom layer, go to point 6, otherwise, go to point 5.

Step 5: It goes down the hierarchy structure by one level. Go to point 2 for those cells that form the relevant cell of the high-level layer.

Step 6: If the specification of the query is met, go to point 8, otherwise go to point 7.

Step 7: Retrieve those data that fall into the relevant cells and do further processing. Return the result that meets the requirement of the query. Go to point 9.

Step 8: Find the regions of relevant cells. Return those regions that meet the requirement of the query. Go to point 9.

Step 9: Stop or terminate.

Advantages:

- Grid-based computing is query-independent because the statistics stored in each cell represent a summary of the data in the grid cells and are query-independent.
- The grid structure facilitates parallel processing and incremental updates.

Disadvantage:

- The main disadvantage of Sting (Statistics Grid). As we know, all cluster boundaries are either horizontal or vertical, so no diagonal boundaries are detected.

What is Outlier in data mining

Whenever we talk about data analysis, the term outliers often come to our mind. As the name suggests, "outliers" refer to the data points that exist outside of what is to be expected. The major thing about the outliers is what you do with them. If you are going to analyze any task to analyze data sets, you will always have some assumptions based on how this data is generated. If you find some data points that are likely to contain some form of error, then these are definitely outliers, and depending on the context, you want to overcome those errors. The data mining process involves the analysis and prediction of data that the data holds. In 1969, Grubbs introduced the first definition of outliers.

Difference between outliers and noise

Any unwanted error occurs in some previously measured variable, or there is any variance in the previously measured variable called noise. Before finding the outliers present in any data set, it is recommended first to remove the noise.

Statistical Distribution-based Outlier Detection

Introduction: The statistical distribution-based approach to outlier detection assumes a distribution or probability model for the given data set (**e.g:** a normal or Poisson distribution) and then identifies outliers with respect to the model using a discordancy test. Application of the test requires knowledge of the data set parameters (such as the assumed data distribution), knowledge of distribution parameters (such as the mean and variance), and the expected number of outliers.

Working of discordancy testing: A statistical discordancy test examines two hypotheses: a working hypothesis and an alternative hypothesis. A working hypothesis, H, is a statement that the entire data set of n objects comes from an initial distribution model, F, that is,

H: $o_i \in F$, where i = 1, 2, . . ., n.

The hypothesis is retained if there is no statistically significant evidence supporting its rejection. A discordancy test verifies whether an object, oi, is significantly large (or small) in relation to the distribution F. Different test statistics have been proposed for use as a discordancy test, depending on the available knowledge of the data. Assuming that some statistic, T, has been chosen for discordancy testing, and the value of the statistic for object oi is vi, then the distribution of T is constructed. Significance probability, SP (vi)= Prob (T > vi), is evaluated. If SP (vi) is sufficiently small, then oi is discordant and the working hypothesis is rejected. An alternative hypothesis, H, which states that oi comes from another distribution model, G, is adopted. The result is very much dependent on which model F is chosen because oi may be an outlier under one model and a perfectly valid value under another.

The alternative distribution is very important in determining the power of the test, that is, the probability that the working hypothesis is rejected when oi is really an outlier. There are different kinds of alternative distributions.

- Inherent alternative distribution: In this case, the working hypothesis that all of the objects come from distribution F is rejected in favor of the alternative hypothesis that all of the objects arise from another distribution, G:
- **H: $o_i \in G$, where i = 1, 2,…., n.**
- F and G may be different distributions or differ only in parameters of the same distribution. There are constraints on the form of the G distribution in that it must have potential to produce outliers. For example, it may have a different mean or dispersion, or a longer tail.
- Mixture alternative distribution: The mixture alternative states that discordant values are not outliers in the F population, but contaminants from some other population, G. In this case, the alternative hypothesis is
- **H: oi ∈ (1-ÊŽ)F ÊŽG, where i = 1, 2, ……, n.**
- Slippage alternative distribution: This alternative states that all of the objects (apart from some prescribed small number) arise independently from the initial model, F, with its given parameters, whereas the remaining objects are independent observations from a modified version of F in which the parameters have been shifted.

There are two basic types of procedures for detecting outliers:

- Block procedures: In this case, either the entire suspect objects are treated as outliers or all of them are accepted as consistent.
- Consecutive (or sequential) procedures: An example of such a procedure is the inside out procedure. Its main idea is that the object that is least “likely” to be an outlier is tested first. If it is found to be an outlier, then all of the more extreme values are also considered outliers; otherwise, the next most extreme object is tested, and so on. This procedure tends to be more effective than block procedures.

Distance-Based Outlier Detection in Data Mining

An outlier is an object that deviates significantly from the rest of the objects. They can be caused by measurement or execution errors. The analysis of outlier data is referred to as outlier analysis or outlier mining.

An outlier is a data object that deviates significantly from the rest of the data objects and behaves in a different manner. An outlier is an object that deviates significantly from the rest of the objects. They can be caused by measurement or execution errors. The analysis of outlier data is referred to as outlier analysis or outlier mining.

Outliers are isolated data objects that do not follow the general trends of data. Outliers cause many problems when we are working in machine learning or deep learning and it also affects the accuracy of the model. So detecting and removing outliers is going to be very important.

Outlier Detection:

Outlier Detection is a natural extension of data mining techniques. As Data Mining is the extraction of general patterns or trends in large datasets, outlier detection is the discovery of data objects that deviate significantly from such general patterns or trends. Such data objects that deviate significantly from other data objects in a dataset are known as outliers.

Finding data objects that are significantly different from other objects is an important activity. By standing out from the crowd, outliers could represent objects that are in some way, much better or much worse than the general trend. They may represent objects that need to be dealt with in some special manner. It is also possible that they represent erroneously entered data or even noise.

Take a look at the best example of how outlier detection help in Data Mining.

- **Fraud detection:** Fraud detection is very important in the modern world. As fraud cases are going increase day by day like a fraud in a credit card transaction, bank loan application, and many more, Outlier detection help us in detecting this fraud as an outlier since they represent instance that deviates from the normal trends.
- **Medicine:** In healthcare detecting outlier play's an important i.e Unusual Symptoms or test results may indicate potential health problems of patients. and there are many other applications of outlier detection in Data Mining.

Distance-Based Outlier Detection Methods

A distance-based outlier detection method consults the neighborhood of an object, which is defined by a given radius. An object is then considered an outlier if its neighborhood does not have enough other points. This is termed as Distance-Based Outlier Detection Methods.

- Distance-Based Methods usually depend on a Multi-dimensional Index, Which is used to retrieve the neighborhood of each object to see if it contains sufficient points. If there are insufficient points, then the object is termed an outlier.
- Distance-Based methods scale better to multi-dimensional space and can be computed more efficiently than the statistical-based method. Identifying Distance-based outliers is an important and useful data mining activity. The main disadvantage of distance-based methods is that distance-based outlier detection is based on a single value of a custom parameter. This can cause significant problems if the dataset contains both dense and sparse regions.

Outlier detection methods can be categorized according to whether the sample of data for analysis is given with expert-provided labels that can be used to build an outlier detection model. In this case, the detection methods are supervised, semi-supervised, or unsupervised. Alternatively, outlier detection methods may be organized according to their assumptions regarding normal objects versus outliers. This categorization includes statistical methods, proximity-based methods, and clustering-based methods.

Algorithms For Mining Distance-Based Outliers:

Below are some algorithms which are used for Mining Distance-Based Outlier more efficiently.

- **Index-based algorithm:** The index-based algorithm facilitates multidimensional indexing structures, including R-trees or k-d trees, to search for neighbors of each object o inside radius d around that object. Once K (K = N(1-p)) neighbors of object o are discovered, it is accessible that o is not an outlier. This algorithm has the lowest case complexity of O ($k * n^2$), where k is the dimensionality, and n is the number of objects in the data set.

- **Nested-loop algorithm:** The nested loop algorithm has the same evaluation complexity as the index-based algorithm but avoids building index structures and minimizes the amount of I/O. It splits the memory buffer in half and puts the data into several logical blocks.
- **Cell-based algorithm:** It avoids the $O(n^2)$ computational complexity and develops a cell-based algorithm for memory-resident datasets. Its complexity is $O(c*k + n)$, where c is a constant based on the number of cells and k is the dimension.

- **Distance-based outlier detection methods:** A distance-based outlier detection method is a statistical technique. Such methods typically measure distances between individual data points and the rest of their respective groups. Many approaches also have a configurable error threshold for determining when a point is an outlier. Many distance-based outliers methods have been developed. The methods use distance statistics such as Euclidean, Manhattan, or Mahalanobis distance for calculating distances between individual points and to detect outliers. The following three outlier detection methods have been selected based on their performance:
    - WLSMV (Weighted Least Squares Minimization) method
    - SVM (Support Vector Machines) method,
    - RMSProp method.

- **Density-based Outlier detection methods:** A density-based outlier detection method is used for checking the density of an entity object and its closest objects. Key applications of this method are used in many applications including Malware Detection, Awareness, Behavior Analysis, and Network Intrusion Detection. There are some limitations to density-based outlier detection methods that are effective until it is determined that the outliers being detected are not necessarily outliers but just a part of a much larger distribution of data. A limitation with using density-based outlier detection methods is that the density function must be defined and clearly understood before implementation and the proper value set.

**********END OF UNIT V**********